Artificial Intelligence CSCC-302

Week-2

Chapter#3

Problem Solving by Searching:

3.1-Problem Solving Agents

3.2-Searching for Solutions

3.3-Uninformed Search Strategies

3.1-Problem Solving Agents

In which we see how an agent can find a sequence of actions that achieves its goals when no single action will do

Goal-based agents that use more advanced factored or structured representations are usually called planning agents

Uninformed and informed search algorithms are two broad categories of search algorithms used in artificial intelligence and computer science to find solutions to problems. They differ primarily in how they explore the search space and make decisions about which paths to explore next.

Uninformed Search Algorithms (Blind Search):

These are search algorithms that operate without any additional information about the problem beyond what is provided in the problem definition. They explore the search space systematically, often without guidance, and make decisions based on basic principles like expanding all possibilities.

1. Breadth-First Search (BFS)

2. Depth-First Search (DFS)

3. Uniform-Cost Search (UCS)

4. Depth-Limited Search (DLS)

5. Iterative Deepening Depth-First Search (IDDFS)

Informed Search Algorithms (Heuristic Search):

These search algorithms, also known as heuristic search algorithms, make use of additional information or heuristics about the problem to guide the search more efficiently. They prioritize paths or nodes that seem more promising based on heuristic estimates and are typically used when some domain-specific knowledge is available.

1. Best-First Search

2. A* Search

3. Greedy Best-First Search

4. IDA* (Iterative-Deepening A*)

5. Hill Climbing

Introduction to Problem-Solving Agents

- Intelligent agents aim to maximize their performance measure.
- Adopting a goal can simplify decision-making for agents.
- Goal formulation is the first step in problem solving.
- Goals help organize behavior by limiting objectives.

Goal Formulation

- Goal is a set of world states where the objective is satisfied.
- Agent needs to decide which actions and states to consider to achieve the goal.
- Problem formulation is the process of defining actions and states for a given goal.
- Problem formulation simplifies the decision-making process.

Search for Solutions

- Once a goal is formulated, the agent needs to find a sequence of actions to achieve it.
- Search involves looking for sequences of actions that lead to states with known values.
- A search algorithm takes a problem as input and returns a solution.
- Execution phase involves carrying out the recommended actions from the solution.
- The process of problem solving follows a "formulate, search, execute" design.

Components of Well-Defined Problems and Solutions

1. Initial State: The state from which the agent starts (e.g., "In Arad" in Romania).

2. Successor Function: Defines possible actions available in a given state.

3. State Space: All states reachable from the initial state, forming a graph.

4. Goal Test: Determines if a state is a goal state.

5. Path Cost Function: Assigns a cost to each path (e.g., travel distance).

Formulating Problems

- Problem formulation involves abstraction.
- Abstraction simplifies the state and action descriptions.
- It involves removing irrelevant details from representations.
- The abstraction must be valid and the abstract actions easy to carry out.
- Abstraction is essential for handling complex real-world scenarios.

Conclusion

1. Problem-solving agents follow a systematic approach to achieve goals.
2. Goal formulation, problem formulation, search for solutions, and execution are key steps.
3. Abstraction helps simplify problem descriptions and make problem-solving feasible in complex environments.

Example Problems in Problem Solving

Toy Problems

Vacuum World

- **States:** Agent in one of two locations, each may or may not contain dirt (8 possible states).
- **Initial State:** Any state can be the initial state.
- **Successor Function:** Generates legal states by performing actions (Left, Right, Suck).
- **Goal Test:** Checks if all squares are clean.
- **Path Cost:** Each step costs 1.

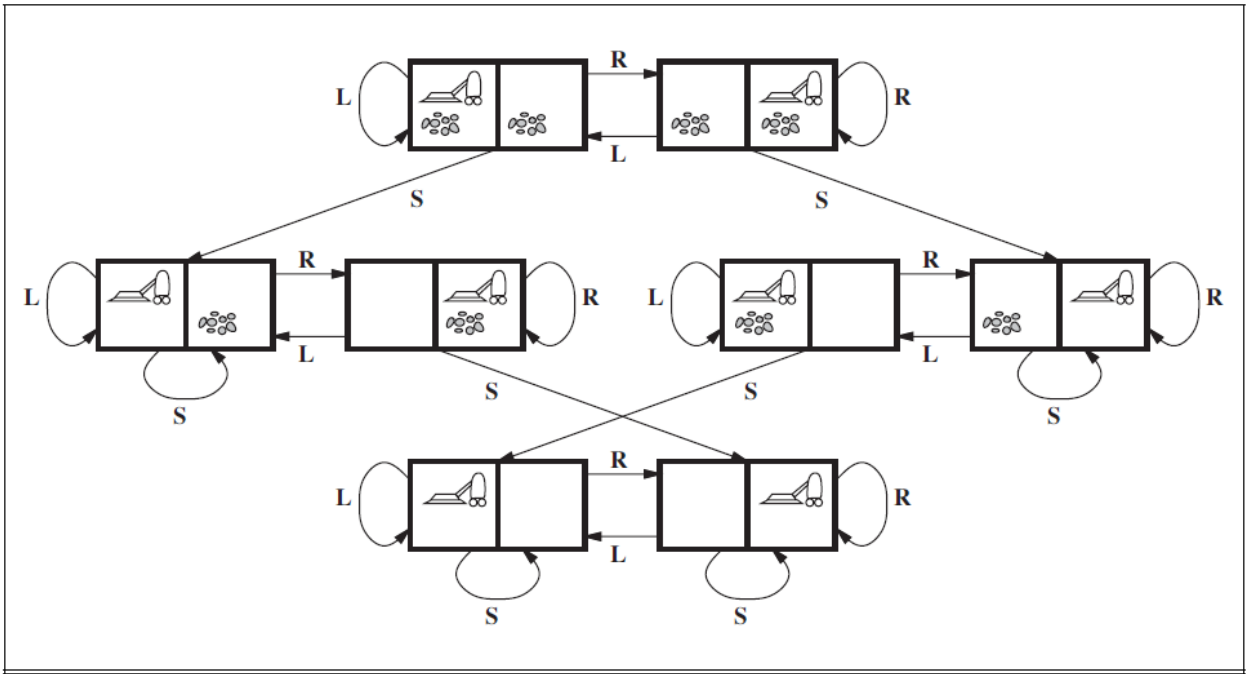

Figure 3.3 The state space for the vacuum world. Links denote actions: L = *Left*, R = *Right*, S = *Suck*.

8-Puzzle

- **States:** 3x3 board with 8 numbered tiles and a blank space.
- **Initial State:** Any state can be the initial state.
- **Successor Function:** Generates legal states by sliding tiles (Left, Right, Up, Down).
- **Goal Test:** Checks if the state matches a specified goal configuration.
- **Path Cost:** Each step costs 1.

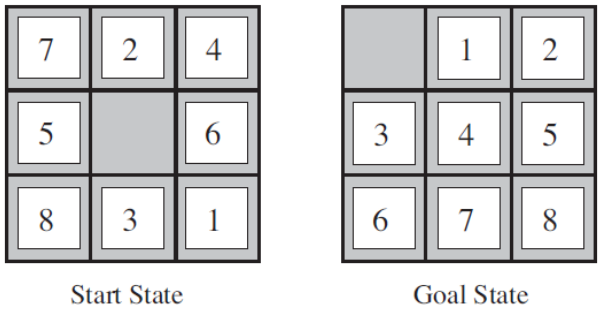

Figure 3.4 A typical instance of the 8-puzzle.

8-Queens Problem

- Goal: Place 8 queens on a chessboard without any of them attacking another.
- Two formulations: incremental (add queens to an empty board) and complete-state (move queens on a board).
- Path cost is not relevant.

Real-World Problems

Route-Finding Problem

- States represent locations and current time.
- Successor function generates states based on scheduled flights.
- Goal test checks if the destination is reached by a specified time.
- Path cost considers various factors such as cost, waiting time, flight time, etc.

Touring Problems

- Include visiting multiple cities while minimizing cost or time.
- States involve the current location and set of visited cities.
- Goal test checks if all cities have been visited.
- Path cost may involve factors like distance or time.

Traveling Salesperson Problem (TSP)

- A variant of touring problems where each city must be visited exactly once.
- Goal: Find the shortest tour.
- NP-hard problem with many applications.

Robot Navigation

- Generalizes route-finding to continuous spaces.
- Robot can have multidimensional movement capabilities.
- Involves dealing with sensor errors and control precision.

Internet Searching

- Involves searching the web for answers, information, or deals.
- Internet conceptualized as a graph of nodes (web pages) connected by links.
- Search techniques play a crucial role in information retrieval.

These problems illustrate the diversity and complexity of real-world and toy problems that can be addressed using problem-solving techniques. Different problems may require specific formulations, search strategies, and optimization criteria based on their unique characteristics and application domains.

3.2-Searching for Solutions

A solution is an action sequence, so search algorithms work by considering various possible action sequences.

The possible action sequences starting at the initial state form a search tree with the initial state at the root.

The branches are actions and the nodes correspond to states in the state space of the problem.

Search algorithms systematically explore the state space of a problem by considering various possible action sequences. These sequences are represented as paths in a search tree, where nodes correspond to states, and branches represent actions. By traversing this tree intelligently, search algorithms aim to find a path from the initial state to a goal state, which constitutes a valid solution to the problem at hand.

Search Tree

- **Search Tree:** Represents the exploration of a problem's state space.
- **Search Node:** Elements of the search tree.
- **Node Expansions:** The process of generating new nodes in the search tree.

Search Strategy

- Search Strategy: Determines the order in which nodes in the search tree are expanded.
- Different strategies can lead to different search tree structures and ultimately different solutions.
- Choice of which state to expand depends on the search strategy.

Node Representation

A node is a data structure with five components:

1. **STATE:** Represents a state in the state space.
2. **PARENT-NODE:** Represents the node in the search tree that generated this node.
3. **ACTION:** Represents the action applied to the parent to generate the node.
4. **PATH-COST:** Indicates the cost of the path from the initial state to the node.
5. **DEPTH:** Represents the number of steps along the path from the initial state.

State Space vs. Search Tree

- **State Space:** Contains the actual configurations of the problem.
- **Search Tree:** Represents the exploration of the state space, including multiple paths to the same state.

Fringe

- Fringe: Collection of nodes that have been generated but not yet expanded.
- Each element of the fringe is a leaf node with no successors in the tree.
- Fringe is essential for determining which node to expand next.
- Fringe can be implemented as a queue.

Queue Operations

- MAKE-QUEUE(element, ...): Creates a queue with the given element(s).
- EMPTY?(queue): Returns true if the queue is empty.
- FIRST(queue): Returns the first element of the queue.
- REMOVE-FIRST(queue): Returns the first element and removes it from the queue.
- INSERT(element, queue): Inserts an element into the queue.
- INSERT-ALL(elements, queue): Inserts a set of elements into the queue.

General Tree-Search Algorithm

```
function TREE-SEARCH(problem, fringe) returns a solution or failure
  fringe <- INSERT(MAKE-NODE(INITIAL-STATE[problem]), fringe)
  loop do
    if EMPTY?(fringe) then return failure
    node <- REMOVE-FIRST(fringe)
    if GOAL-TEST[problem] applied to STATE[node] succeeds
      then return SOLUTION(node)
    fringe <- INSERT-ALL(EXPAND(node, problem), fringe)
```

Infrastructure for search algorithms

Infrastructure for search algorithms is essential to keep track of the search tree and efficiently explore the state space. This infrastructure typically involves data structures associated with each node in the search tree.

Let's delve into the four components of this infrastructure

1. n.STATE:

- This component represents the state in the state space that the node corresponds to.
- The state describes the configuration or condition of the problem at a particular point in the search.

2. n.PARENT:

- The "parent" component of a node points to the node in the search tree that generated the current node.
- It establishes the link between the current node and its predecessor in the search process.

3. n.ACTION:

- This component records the action that was applied to the parent node to generate the current node.
- Actions represent the specific steps or moves taken to transition from one state to another.
- It helps in reconstructing the sequence of actions leading from the initial state to a solution.

4. n.PATH-COST (traditionally denoted as g(n)):

- The path cost component represents the cost associated with the path from the initial state to the current node.
- It accumulates the cost of actions or transitions taken along the path.
- This information is crucial for determining the quality or optimality of a solution.

These components collectively enable search algorithms to navigate through the state space efficiently and make informed decisions about which nodes to explore next. The parent-child relationships in the search tree, along with action and path cost information, allow algorithms to trace back to the initial state and find the optimal solution when applicable.

The infrastructure for search algorithms is adaptable and can be implemented using various data structures, such as nodes in a tree or elements in a priority queue, depending on the specific algorithm and problem domain. It forms the foundation for informed exploration of the search space, helping algorithms find solutions while managing computational resources effectively.

Measuring Problem-Solving Performance

Evaluation of algorithm performance includes:

1. Completeness: Does the algorithm always find a solution when one exists?
2. Optimality: Does the algorithm find the optimal solution?
3. Time Complexity: How long does it take to find a solution?
4. Space Complexity: How much memory is needed for the search?

Complexity Measures

- In theoretical computer science, complexity is measured in terms of the size of the state space graph.
- In AI, complexity is expressed in terms of branching factor (b), depth of shallowest goal node (d), and maximum path length (m).
- Time complexity is often measured by the number of nodes generated during the search.
- Space complexity is measured by the maximum number of nodes stored in memory.

Total Cost

- Total cost combines search cost and path cost.
- It is essential when evaluating algorithms that consider both time and path length.
- Total cost allows for trade-offs between different criteria, such as time and distance, to find an optimal solution.

The choice of search strategy, representation, and evaluation measures depends on the specific problem and its characteristics. Different problems may require different strategies to achieve the desired outcomes effectively and efficiently.

3.3-Uninformed Search Strategies

UNINFORMED SEARCH STRATEGIES

Uninformed search, also known as blind search, refers to a category of search strategies that operate solely based on the information provided in the problem definition. These strategies have no additional knowledge about the states they encounter beyond what is explicitly defined. Here, we will explore five common uninformed search strategies.

1. Breadth-First Search:

- In breadth-first search, the root node is expanded first, followed by all its successors, then their successors, and so on.
- This strategy explores all nodes at a particular depth level before moving on to the next level.
- It can be implemented using a FIFO (First-In-First-Out) queue, ensuring that shallower nodes are explored before deeper ones.

2. Uniform-Cost Search:

- Uniform-cost search expands the node with the lowest path cost, considering the cumulative cost of reaching that node from the initial state.

- It is similar to breadth-first search when all step costs are equal but can handle variable step costs.
- To guarantee completeness and optimality, step costs should be greater than or equal to a small positive constant.

3. Depth-First Search:

- Depth-first search explores the deepest node in the current fringe before backtracking to shallower nodes.
- It is typically implemented using a Last-In-First-Out (LIFO) queue or a recursive approach.
- Depth-first search has minimal memory requirements, storing only a single path from the root to a leaf node and its unexpanded siblings.

4. Depth-Limited Search:

- Depth-limited search is a variation of depth-first search that limits the maximum depth it explores.
- This strategy addresses the infinite-path problem but may fail to find solutions if the depth limit is set too shallow.

5. Iterative Deepening Depth-First Search (Iterative Deepening Search):

- Iterative deepening search combines the benefits of depth-first and breadth-first search.
- It starts with a depth limit of 0, gradually increasing the limit until a solution is found.
- This approach ensures completeness, optimality, and moderate memory requirements.

6. Bidirectional Search:

- Bidirectional search runs two simultaneous searches: one forward from the initial state and one backward from the goal state.
- The searches continue until they meet in the middle, indicating a solution.
- Bidirectional search reduces the time complexity significantly but may require additional considerations, especially when searching backward from the goal.

Each of these uninformed search strategies has its advantages and disadvantages. The choice of which strategy to use depends on factors such as the problem domain, the available computational resources, and the characteristics of the state space. Uninformed search strategies are often suitable for smaller state spaces or as a baseline before exploring more advanced search techniques that incorporate heuristic information (informed search strategies).